Camera Calibration Guide

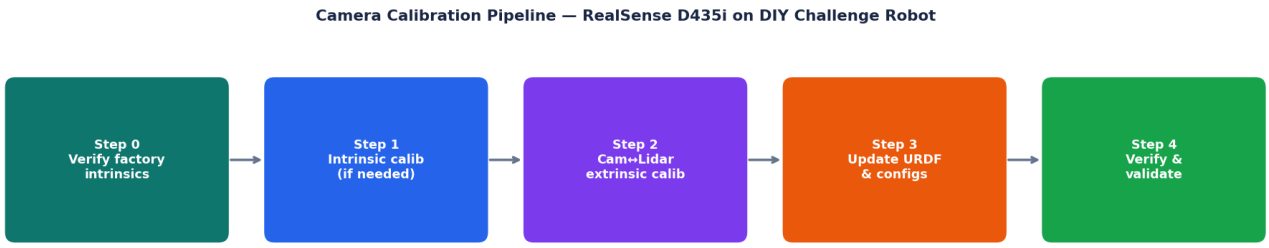
Intel RealSense D435i — Intrinsic, Extrinsic & Camera-to-LiDAR Calibration

DIY Challenge Repo · ROS 2 Humble · Hesai QT64 · Jetson Nano

Camera	Intel RealSense D435i (stereo IR + RGB + IMU)
LiDAR	Hesai QT64 3D lidar
Compute	NVIDIA Jetson Nano 4 GB / JetPack 5.x
ROS 2 Version	Humble
Intrinsic source	Factory EEPROM (auto-loaded by RealSense driver) — manual recal optional
Extrinsic (cam↔lidar)	Must be measured/calibrated — placeholder values in URDF must be replaced

0. Overview — What Calibration Is Needed?

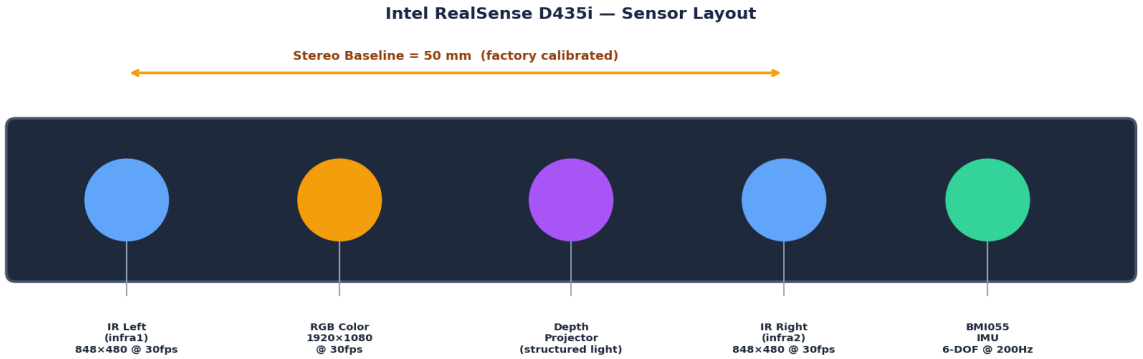
The RealSense D435i has two distinct calibration requirements that serve completely different purposes. Understanding which ones you need — and which are already handled — is essential before touching any config files.



Calibration Type	What it provides	Already done?	Action required
Intrinsic (per camera)	fx, fy, cx, cy, distortion coefficients	YES — factory calibrated, stored in EEPROM	Verify. Recalibrate only if camera takes a hard knock or intrinsics look wrong.
Stereo baseline (IR left ↔ IR right)	50 mm baseline transform between the two IR cameras	YES — factory calibrated	None. Never attempt to re-do stereo extrinsics manually.
Camera ↔ LiDAR extrinsic	6-DOF transform from camera_link to lidar_link	NO — URDF has placeholder values (CALIB: markers)	REQUIRED. Measure physically + optionally refine with target-based calibration. This is the critical step.
Camera ↔ Robot (URDF TF)	base_link → camera_link static transform	Partial — placeholder xyz="0.20 0.0 0.20"	Update URDF with physically measured or cam-lidar-calibrated values.
Time synchronisation	Align camera and lidar timestamps	Partial — RealSense driver handles internal sync	Verify clock offset between camera and lidar host (chronyd or PTP recommended).

WARNING: The URDF file (src/diy_robot_description/urdf/robot.urdf.xacro) currently has PLACEHOLDER values for camera_link (xyz="0.20 0.0 0.20"). These must be replaced with real measured values before the robot is used. Using wrong extrinsics will cause misaligned sensor fusion and poor localization.

1. Understanding Camera Intrinsic Parameters



1.1 What are intrinsic parameters?

Intrinsic parameters describe the internal optics of a single camera — they are independent of where the camera is mounted. They are captured in the 3×3 camera matrix K:

Camera intrinsic matrix

```
K = | fx   0   cx |
    |  0  fy   cy |
    |  0   0    1 |

fx, fy  — focal lengths in pixels (how much the lens magnifies the scene)
cx, cy  — principal point (pixel coordinates of the optical axis intersection)
          Ideally: cx ≈ image_width/2,  cy ≈ image_height/2

Distortion coefficients (Brown-Conrady model):
  [k1, k2, p1, p2, k3]
  k1, k2, k3 — radial distortion  (barrel / pincushion)
  p1, p2      — tangential distortion (lens not perfectly parallel to sensor)
```

1.2 D435i factory intrinsics — what you already have

The RealSense D435i stores its calibration data in on-board EEPROM, factory-calibrated by Intel at sub-pixel precision. The realsense2_camera ROS 2 driver reads this data at startup and publishes it automatically on the camera_info topics:

- /camera/color/camera_info — RGB colour camera intrinsics
- /camera/infra1/camera_info — Left IR camera intrinsics
- /camera/infra2/camera_info — Right IR camera intrinsics
- /camera/depth/camera_info — Depth stream intrinsics

TIP: For this competition stack you do NOT need to do intrinsic calibration. The factory calibration is sufficient for YOLO detection, depth sensing, and stereo VSLAM. Proceed directly to Section 3 (camera-to-lidar extrinsic).

1.3 How to inspect the factory intrinsics

With the camera plugged in and the RealSense driver running, inspect the published calibration:

Inspect camera intrinsics from ROS 2 topic

```
# Start the RealSense driver (or the full bringup)
ros2 launch realsense2_camera rs_launch.py

# In another terminal - print the colour camera intrinsics
ros2 topic echo /camera/color/camera_info --once

# Expected output (approximate for D435i @ 1280x720):
```

```
# width: 1280
# height: 720
# K: [911.3, 0, 640.5, 0, 911.3, 360.3, 0, 0, 1]
#        fx       cx        fy       cy
# D: [-0.054, 0.068, 0.001, -0.001, -0.022]   (distortion k1..k3, p1, p2)
# distortion_model: plumb_bob
```

1.4 Typical D435i factory values

Stream	Resolution	fx ≈ fy	cx ≈ w/2	cy ≈ h/2
IR (infra1/2)	848 × 480	~430 px	~424 px	~240 px
Color (RGB)	1280 × 720	~911 px	~641 px	~360 px
Color (RGB)	640 × 480	~606 px	~321 px	~240 px

NOTE: If the published fx/fy values differ significantly from the table above (e.g. fx < 300 or > 1100 for RGB), or if cx/cy are far from image centre, the EEPROM data may be corrupted. In that case proceed to Section 2 for manual recalibration.

2. Manual Intrinsic Calibration (Only If Needed)

WARNING: Only perform this section if Section 1.3 shows corrupted or clearly wrong factory intrinsics. The factory calibration is typically more accurate than a manual checkerboard calibration.

2.1 What you need

- **Checkerboard pattern**: 9×6 inner corners, 25 mm square size recommended (print on A3 paper, mount flat on rigid board — no curling)
- **ROS 2 camera_calibration package**: sudo apt install ros-humble-camera-calibration
- **Good lighting**: bright, diffuse — no shadows on the checkerboard
- **~50–70 captures**: varying distance (0.5–2 m), tilt, and rotation

2.2 Run the calibration

Install and run camera_calibration for the colour camera

```
# Install if not present
sudo apt install ros-humble-camera-calibration

# Start the RealSense driver in another terminal
ros2 launch realsense2_camera rs_launch.py  color_width:=640 color_height:=480

# Run the calibrator (9x6 inner corners, 0.025 m square size)
ros2 run camera_calibration cameracalibrator \
  --size 9x6 \
  --square 0.025 \
  --ros-args \
  --remap image:=/camera/color/image_raw \
  --remap camera:=/camera/color

# A GUI window opens. Move the checkerboard until all four bars fill:
#   X — horizontal coverage (slide board left and right across full frame)
#   Y — vertical coverage   (slide board up and down across full frame)
#   Size — distance variation (move board closer and farther, 0.5–2 m)
#   Skew — tilt variation   (tilt board ±30° in all axes)
# Once fully green, click CALIBRATE, then SAVE.
# Output saved to /tmp/calibrationdata.tar.gz
```

For IR cameras (used in stereo VSLAM)

```
# Left IR camera only (use this if recalibrating for VSLAM)
ros2 run camera_calibration cameracalibrator \
  --size 9x6 \
  --square 0.025 \
  --ros-args \
  --remap image:=/camera/infra1/image_rect_raw \
  --remap camera:=/camera/infra1

# NOTE: Disable the IR emitter first to get a clean passive image:
# ros2 param set /realsense2_camera enable_infra_emitter false
```

2.3 Apply the new calibration

Extract and apply calibration YAML

```
# Extract the calibration archive
cd /tmp && tar -xzf calibrationdata.tar.gz
# This creates: ost.yaml  (the new intrinsics)

# The RealSense driver can load a custom calibration file:
```

```
ros2 launch realsense2_camera rs_launch.py \
  color_camera_info_url:=file:///tmp/ost.yaml

# To make it permanent, copy to the config directory:
cp /tmp/ost.yaml ~/ros2_ws/src/DIY-Challenge-Repo/src/challenge_bringup/config/color_camera_info.yaml
# Then update challenge_master.launch.py to pass color_camera_info_url
```

NOTE: The RealSense driver's camera_info_url parameter overrides the EEPROM calibration for the specified stream. Other streams continue to use factory calibration. Calibration scripts are provided in scripts/calibrate_camera_intrinsics.sh.

3. Camera ↔ LiDAR Extrinsic Calibration (Required)

This is the **most important calibration step** for this robot. The extrinsic calibration defines the 6-DOF rigid-body transform between camera_link and lidar_link. Without it, any node that fuses camera and lidar data (or uses the camera relative to the map) will have incorrect spatial reasoning.

WARNING: The current URDF has placeholder values: base_to_camera xyz="0.20 0.0 0.20". These are a rough estimate only. They must be replaced with real measured values before competition.

3.1 What the extrinsic transform is

Extrinsic transform — what it means

```
# The transform T_lidar_camera answers:
#   "Given a 3D point in camera_link frame, where is it in lidar_link frame?"
#
# It is a 4x4 homogeneous matrix:
#   T = | R  t |    R: 3x3 rotation matrix
#       | 0  1 |    t: 3x1 translation vector (x, y, z in metres)
#
# This is what goes into robot.urdf.xacro:
#   <origin xyz="tx ty tz" rpy="roll pitch yaw"/>
# under the base_to_camera joint.
#
# It is also written to the nav2_params.yaml obstacle source transform
# if camera detections are fused into the costmap.
```

3.2 Method A — Physical measurement (start here)

For competition purposes, a careful physical measurement is sufficient as a starting point. Use a ruler or tape measure to determine the 3D offset of the camera lens centre from the lidar origin.

How to physically measure camera-lidar offset

```
# Reference frames:
#   lidar_link origin  = centre of Hesai QT64 rotating head
#   camera_link origin = centre of the D435i lens array

# Measure in ROBOT body frame (base_link convention: x=forward, y=left, z=up):

# 1. x offset: horizontal distance FORWARD from lidar to camera
#    (positive if camera is in front of lidar)
x = <measure in metres, e.g. 0.12>

# 2. y offset: lateral distance (positive = camera to LEFT of lidar)
#    (0.0 if both are on the robot centreline)
y = <measure in metres, e.g. 0.0>

# 3. z offset: vertical distance UP from lidar to camera
#    (positive if camera is above lidar)
z = <measure in metres, e.g. -0.05>

# 4. rpy: rotation of camera frame relative to lidar frame
#    For a camera mounted level and facing forward, same as lidar:
#    rpy = "0 0 0"  (no rotation)
#    If camera is tilted down 10 degrees to see ground objects:
#    rpy = "0 0.175 0"  (0, 10°, 0 in radians)

# Update robot.urdf.xacro:
# <origin xyz="0.12 0.0 -0.05" rpy="0 0 0"/>  ← under base_to_camera joint
```

3.3 Method B — Target-based calibration with Kalibr (precision)

Kalibr is the industry-standard open-source tool for multi-sensor calibration. It uses an April-Tag target board to jointly solve camera intrinsics, camera-IMU extrinsics, and multi-camera geometry. For camera-to-lidar it is used alongside a known flat calibration target visible to both sensors.

- Accuracy: ~1–3 mm translation, ~0.1° rotation (vs ~5–10 mm for manual measure)
- Time: ~2 hours for full setup, data collection, and solving
- Recommended if VSLAM or precise sensor fusion is required for the competition
- Not required if the robot only uses the camera for YOLO detection (coarse alignment sufficient)

Install Kalibr (Ubuntu 22.04 / ROS 2 Humble)

```
# Kalibr runs best in a Docker container on Ubuntu 22.04
docker pull ethzasl/kalibr:latest

# Or build from source:
cd ~/  &&  git clone https://github.com/ethz-asl/kalibr.git
cd kalibr
# Follow https://github.com/ethz-asl/kalibr/wiki/installation for ROS 2
```

Camera-to-lidar calibration data collection

```
# You need a calibration target visible in BOTH the camera AND the lidar.
# Best approach: a flat board with AprilTags + lidar-reflective markers.
#
# Step 1: Record a ROS 2 bag with both sensors:
ros2 bag record /camera/color/image_raw /camera/color/camera_info \
                /hesai/points \
                -o ~/calib_cam_lidar_$(date +%Y%m%d_%H%M%S)
#
# Step 2: Move the target board to 15-20 different positions/orientations
#         covering the overlapping FOV of camera and lidar.
#         Keep the robot stationary during each capture.
#
# Step 3: Run the Kalibr camera-lidar solver:
#   See: https://github.com/ethz-asl/kalibr/wiki/camera-imu-calibration
#   (adapt for lidar as the second sensor)
```

3.4 Method C — scripts/calibrate_cam_lidar.sh helper

The repo provides a helper script that automates the data collection step. Run it on the robot, then process the bag offline:

Use the calibration helper script

```
# Ensure both camera and lidar are running, then:
./scripts/calibrate_cam_lidar.sh

# The script:
#   1. Verifies /camera/color/image_raw and /hesai/points are publishing
#   2. Records a ~3-minute bag to calibration/cam_lidar_<timestamp>/
#   3. Prints instructions for processing with Kalibr offline
#
# After processing, update the URDF with the result (Section 4).
```

4. Applying Calibration Results to the Repository

4.1 Update the URDF camera joint

Open src/diy_robot_description/urdf/robot.urdf.xacro and replace the placeholder camera joint origin with your measured values:

robot.urdf.xacro — replace placeholder (line ~118)

```
<!-- BEFORE (placeholder): -->
<joint name="base_to_camera" type="fixed">
  <parent link="base_link"/>
  <child link="camera_link"/>
  <origin xyz="0.20 0.0 0.20" rpy="0.0 0.0 0.0"/>   <!-- ← PLACEHOLDER -->
</joint>

<!-- AFTER (example with measured values): -->
<joint name="base_to_camera" type="fixed">
  <parent link="base_link"/>
  <child link="camera_link"/>
  <origin xyz="0.18 0.0 0.22" rpy="0.0 0.05 0.0"/>  <!-- ← YOUR VALUES -->
</joint>
#
# xyz = translation in metres (x forward, y left, z up)
# rpy = roll pitch yaw in radians
#       e.g. rpy="0 0.05 0" tilts camera slightly downward (pitch 3°)
```

4.2 Rebuild the robot description package

Rebuild and verify TF tree

```
# Rebuild after URDF change
cd ~/ros2_ws && colcon build --packages-select diy_robot_description

# Verify the TF tree is correct
ros2 launch diy_robot_description description.launch.py &
ros2 run tf2_tools view_frames  # saves frames.pdf
# Check that base_link → camera_link transform matches your measurements
```

4.3 Verify with RViz2

Visual verification in RViz2

```
# Launch the full robot stack
ros2 launch challenge_bringup challenge_master.launch.py

# Open RViz2, add:
#   - PointCloud2 display: topic /hesai/points, fixed frame: base_link
#   - Image display: topic /camera/color/image_raw
#   - TF display (shows all coordinate frames)
#
# Place a known object (e.g. a 30cm box) at a measured distance in FRONT of the robot.
# Confirm:
#   - The lidar point cloud shows the box at the correct distance
#   - The camera image shows the box
#   - The TF arrow from base_link to camera_link points to the correct location on the robot
```

4.4 Verify sensor alignment numerically

Check timestamp synchronisation and topic rates

```
# Check that camera and lidar are publishing at expected rates
ros2 topic hz /camera/color/image_raw   # expect ~30 Hz
ros2 topic hz /hesai/points             # expect ~10-20 Hz
```

```
# Check timestamp alignment (both should show recent, matching stamps)
ros2 topic echo /camera/color/image_raw --field header.stamp --once
ros2 topic echo /hesai/points --field header.stamp --once

# Large timestamp offset (>100 ms) indicates clock sync issue.
# Fix: ensure both camera and Jetson are NTP/PTP synchronised:
sudo chronyc tracking   # check NTP sync status
```

5. Time Synchronisation Between Camera and LiDAR

The RealSense D435i uses its own internal clock. The Hesai QT64 lidar has its own clock. If the Jetson Nano is not time-synchronised, timestamps can diverge by hundreds of milliseconds, corrupting any time-based sensor fusion.

5.1 Check current clock status

Verify system clock is synchronised

```
# Check NTP synchronisation status
timedatectl status
# Look for: "System clock synchronized: yes"
# If not synchronized, install chrony:
sudo apt install chrony
sudo systemctl enable --now chrony
sudo chronyc makestep  # force immediate sync
```

5.2 RealSense hardware timestamp

Configure RealSense driver to use hardware timestamps

```
# In challenge_master.launch.py, the realsense2_camera node already runs.
# To use hardware timestamps (lower jitter):
ros2 launch realsense2_camera rs_launch.py \
  enable_sync:=true \
  unite_imu_method:=linear_interpolation

# The enable_sync parameter synchronises all RealSense streams to the same
# hardware timestamp, reducing inter-stream offset to <1 ms.
```

5.3 Lidar-camera timestamp offset check

Measure actual timestamp offset

```
# Record one message from each sensor and compare stamps
python3 - <<'EOF'
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import PointCloud2, Image
import time

class StampCheck(Node):
    def __init__(self):
        super().__init__('stamp_check')
        self.lidar_stamp = None
        self.camera_stamp = None
        self.create_subscription(PointCloud2, '/hesai/points',
            lambda m: setattr(self, 'lidar_stamp', m.header.stamp), 1)
        self.create_subscription(Image, '/camera/color/image_raw',
            lambda m: setattr(self, 'camera_stamp', m.header.stamp), 1)

rclpy.init()
node = StampCheck()
rclpy.spin_once(node, timeout_sec=2)
rclpy.spin_once(node, timeout_sec=2)
if node.lidar_stamp and node.camera_stamp:
    diff = abs((node.lidar_stamp.sec + node.lidar_stamp.nanosec*1e-9) -
               (node.camera_stamp.sec + node.camera_stamp.nanosec*1e-9))
    print(f"Timestamp offset: {diff*1000:.1f} ms")
    print("OK" if diff < 0.05 else "WARNING: offset > 50ms, check NTP sync")
EOF
```

6. Camera Setup Checklist

Complete all items in order before the competition. Items marked REQUIRED must be done. Items marked OPTIONAL improve accuracy.

	Task	Priority	How to verify
■	Plugin camera and confirm RealSense driver starts without errors	REQUIRED	ros2 topic list | grep camera — should show 10+ topics
■	Inspect factory intrinsics (Section 1.3) — check fx/fy are reasonable	REQUIRED	ros2 topic echo /camera/color/camera_info --once
■	Physically measure camera-to-lidar offset (Section 3.2)	REQUIRED	Ruler/tape measure, record x/y/z to nearest 5 mm
■	Update robot.urdf.xacro base_to_camera joint with measured values	REQUIRED	grep "base_to_camera" urdf/robot.urdf.xacro — no CALIB: comment
■	Rebuild diy_robot_description package	REQUIRED	colcon build --packages-select diy_robot_description
■	Verify TF tree in RViz2 (camera_link at expected position)	REQUIRED	ros2 run tf2_tools view_frames → inspect frames.pdf
■	Verify NTP clock synchronisation on Jetson	REQUIRED	timedatectl status → "System clock synchronized: yes"
■	Run timestamp offset check (Section 5.3) — offset < 50 ms	REQUIRED	Run python3 stamp check script — prints offset in ms
■	Manual intrinsic recalibration if factory values look wrong	OPTIONAL	scripts/calibrate_camera_intrinsics.sh (Section 2)
■	Kalibr precision cam-lidar extrinsic calibration	OPTIONAL	scripts/calibrate_cam_lidar.sh + offline Kalibr processing
■	Enable RealSense hardware sync (enable_sync:=true)	OPTIONAL	ros2 topic hz /camera/color/image_raw — stable 30 Hz

Quick reference — files to edit

File	What to change	When
src/diy_robot_description/urdf/robot.urdf.xacro	base_to_camera joint xyz/rpy — replace placeholder	After physical measurement (required)
src/challenge_bringup/config/color_camera_info.yaml (create new)	Custom intrinsic calibration YAML from camera_calibration	Only if manual intrinsic recalibration done
src/challenge_bringup/launch/challenge_master.launch.py	Add color_camera_info_url param to realsense2 node	Only if custom intrinsics file created above

Further reading

- Intel RealSense ROS 2 wrapper: https://github.com/IntelRealSense/realsense-ros

- ROS 2 camera_calibration package: https://github.com/ros-perception/image_pipeline/tree/humble/camera_calibration
- Kalibr multi-camera calibration: https://github.com/ethz-asl/kalibr
- Intel RealSense SDK calibration tool: rs-calibrate (included in librealsense)
- ROS 2 TF2 tutorial: https://docs.ros.org/en/humble/Tutorials/Intermediate/Tf2/Introduction-To-Tf2.html